

Vorlesung "Softwaretechnik"

Anwendungsfälle (Use Cases)

Lutz Prechelt

Freie Universität Berlin, Institut für Informatik

- Was ist ein Use Case?
 - Beispiele
- Wichtige Parameter
 - Bereich ("system under discussion")
 - Detailgrad/Zielniveau
- Konventionell vs. Agil
- Schrittweise Präzisierung
 - Gefahren von Präzision
- Use-Case-Hierarchien
 - Überblick, Benutzerziele, Details
 - Warum-Wie-Verbindung
- Gesamtverfahren, Checkliste

Wo sind wir?: Taxonomie "Die Welt der Softwaretechnik"

Welt der Problemstellungen:

- Produkt (Komplexitätsprob.)
 - Anforderungen (Problemraum)
 - Entwurf (Lösungsraum)
- Prozess (psycho-soziale P.)
 - Kognitive Beschränkungen
 - Mängel der Urteilskraft
 - Kommunikation, Koordination
 - Gruppendynamik
 - Verborgene Ziele
 - Fehler

Welt der Lösungsansätze:

- Technische Ansätze ("hart")
 - Abstraktion
 - Wiederverwendung
 - Automatisierung
- **Methodische Ansätze ("weich")**
 - **Anforderungsermittlung**
 - **Entwurf**
 - **Qualitätssicherung**
 - **Projektmanagement**

- Einsicht: Man darf sich nicht auf intuitiven Eindruck darüber verlassen, was gebaut werden sollte
 - sondern sollte die Anforderungen systematisch ermitteln
- Prinzipien:
 - **Erhebung** der Anforderungen bei allen Gruppen von Beteiligten
 - **Beschreibung** in einer Form, die die Beteiligten verstehen
 - **Validierung** anhand der verschriftlichten Form
 - **Spezifikation**: Übertragung in zur Weiterverarbeitung günstige Form
 - **Trennung von Belangen**: Anford. möglichst wenig koppeln
 - **Analyse auf Vollständigkeit**: Lücken aufdecken und schließen
 - **Analyse auf Konsistenz**: Widersprüche aufdecken und lösen
 - **Mediation**: Widersprüche, die auf Interessengegensätzen beruhen, einer Lösung zuführen (Kompromiss oder Win-Win)
 - **Verwaltung**: Übermäßige Anforderungsänderungen eindämmen, Anforderungsdokument immer aktuell halten

Was ist ein Use Case (Anwendungsfall)?

- Rolle:
 - Vereinbarung zwischen den Beteiligten eines Projekts über das Verhalten des Systems
- Ansatz:
 - **Beschreibung** des **Verhaltens** des **Systems**
 - als Antwort auf einen Auftrag eines **Hauptakteurs**,
 - der mit seinem Auftrag ein **Ziel** verfolgt und erreicht
- Ein Use Case bündelt die Beschreibungen für viele einzelne Fälle (Szenarios), die den Zielen nach eng zusammen gehören
 - ist also abstrakt
 - aber nicht sehr

Welche Form hat ein Use Case?

- Normalerweise Text
 - meist strukturierter Text
- Das gleiche Prinzip (siehe später) ist aber auch mit formalen, insbesondere grafischen Notationen anwendbar:
 - Flussdiagramme
 - Zustandsdiagramme
 - Sequenzdiagramme
 - Petrinetze u.a.
- Text ist aber meist die beste Wahl
- Anmerkung: Wer nicht gut Text schreiben kann, kann auch nicht gut Use Cases schreiben

Beispiel: Use Case "Über WWW Aktien kaufen"

- **Hauptakteur:** Käufer
- **Anwendungsbereich:**
 - Personal Advisors/Finance package (**PAF**)
- **Niveau:** Benutzerziel
- **Beteiligte und Interessen:**
 - Käufer: Will Aktien kaufen und in sein PAF Portfolio ergänzen
 - Broker: Benötigt vollständige Kaufinformation
- **Voraussetzung:** Benutzer hat PAF gestartet
- **Mindestzusicherung:**
 - Es wird genügend Protokollinformation gesammelt, damit PAF ggf. entdecken kann, wenn etwas schiefgeht um dann beim Benutzer nachzufragen
- **Zusicherung im Erfolgsfall:**
 - Broker-Website hat den Kauf bestätigt; Protokolle u. Portfolio sind aktualisiert

The screenshot shows the 'comdirect' 'Demokonto' interface. At the top, there's a navigation bar with 'logout', 'Kursuche', and 'Volle Suche'. Below it, a 'Persönlicher Bereich' (Personal Area) with links to 'Informen', 'Produkte', 'Services', and 'Forum'. A secondary bar contains 'Depot', 'Order', 'Konto', 'Geldanlage', 'Beratung', 'Musterdepot', 'Postbox', and 'Verwaltung'. The main content area is titled 'INLANDSORDER' and displays user information for 'Max Mustermann' (Kundennummer: 11111111 / BLZ: 20041111), 'Verfügbare Betrag: 6.740,00 EUR', and 'Kontostand: 8.000,00 EUR'. To the right, there's a sidebar with buttons for 'Inlandsorder', 'LiveTrading', 'US-Order', 'Fondsorder', 'Kombinationsorder', 'Orderbuch', 'Ordervorlagen', 'Neuemissionen', and 'Kapitalmaßnahmen'. The main form, 'Angaben zur Order', is on step 1 of 2. It includes fields for 'Inlandsorder' (radio buttons for 'Kauf' and 'Verkauf'), 'ISIN/WKN' (803200), 'Bezeichnung' (Wird automatisch eingetragen), 'Stück/Nominale' (100), 'Börse' (Bitte wählen Sie), 'Limit' (radio buttons for 'billigst / bestens' and 'Limit'), and 'Gültigkeit' (radio buttons for 'tagesgültig' and 'ultimo' with a date selector for 'Januar'). At the bottom, there are buttons for 'Abbrechen', 'Ordervorlage speichern', and 'Weiter'.

Bsp: Use Case

"Über WWW Aktien kaufen" (2)

- **Haupt-Erfolgsszenario:**

1. Käufer wählt "Aktien über das WWW kaufen" aus
2. PAF fordert Name der Website vom Käufer an
3. PAF öffnet Website, behält aber die Kontrolle
4. Käufer benutzt die Website zum Aktienkauf
5. PAF fängt die Reaktionen der Website ab und aktualisiert des Käufers Portfolio
6. PAF zeigt dem Käufer das neue Portfolio

Schritt evtl. nur angedeutet

2 Akteure

Akteur ist stets aktiv

Ziele sind erkennbar

Akteur = Rolle

Bsp: Use Case

"Über WWW Aktien kaufen" (3)

- **Erweiterungen:**

- 3a. Web-Fehler beim Verbindungsaufbau
 - 3a1. PAF meldet Fehler an Benutzer (mit Ratschlag) und geht zurück zum vorherigen Schritt
 - 3a2. Käufer verlässt Use Case oder probiert erneut
- 4a. Computer stürzt während Transaktion ab oder wird ausgeschaltet
 - 4a1. (Was machen wir da???)
- 4b. Website bestätigt Kauf nicht sofort, sondern stellt ihn auf Warteliste
 - 4b1. PAF protokolliert dies und setzt einen Terminmarker, um den Käufer später nach dem Ergebnis zu fragen
- ...
- 5a. Website liefert zu wenig Information über den Kauf
 - 5a1. PAF protokolliert den Mangel, lässt Käufer Kauf aktualisieren

0, 1 oder mehrere
Varianten pro Schritt

Use Cases müssen nicht
immer vollständig sein

Verweis auf
Unter-Use-Case

Ein realer Use Case

- Die Beschreibung auf den drei vorherigen Folien war übrigens ein realer Use Case aus einem realen Projekt
Und er ist recht gut gelungen:
 - Enthält genügend detaillierte Information
 - Ziel, Randbedingungen, Schritte, Problemsituationen
 - aber kaum etwas Unnötiges

Beispiel 2: Auto-Unfallschaden ersetzt bekommen

- **Hauptakteur:** Geschädigter
- **Anwendungsbereich:** SicherMich AG (Versicherung)
- **Niveau:** Überblick
- **Beteiligte und Interessen:**
 - Geschädigter: Möglichst hohe Erstattung bekommen
 - SicherMich: Möglichst wenig Erstattung bezahlen
 - Aufsichtsbehörde: Einhaltung der Gesetze sicherstellen
- **Voraussetzung:** --
- **Mindestzusicherung:**
 - SicherMich protokolliert Schadensmeldung und alle Aktivitäten
- **Zusicherung im Erfolgsfall:**
 - Geschädigter u. SicherMich einigen sich über Entschädigung und SicherMich zahlt diesen Betrag an Geschädigten
- **Auslöser:** Geschädigter reicht Schadensmeldung ein



Beispiel 2: Auto-Unfallschaden ersetzt bekommen (2)

- **Haupt-Erfolgsszenario:**

1. Geschädigter reicht Schadensmeldung und Detailinformation ein
2. SicherMich prüft, dass Geschädigter eine gültige Police hat
3. SicherMich weist den Fall einem Sachbearbeiter zur Prüfung zu
4. SicherMich stellt fest, dass der Sachverhalt gemäß der Versicherungsbedingungen versichert ist
5. SicherMich zahlt an Geschädigten und schließt den Fall



egal ob manuell oder
mittels einer Software

Beispiel 2: Auto-Unfallschaden ersetzt bekommen (3)

- **Erweiterungen:**

- 1a. Eingereichte Daten sind unvollständig
 - 1a1. SicherMich bittet Geschädigten um Auskünfte
 - 1a2. Geschädigter liefert fehlende Informationen
- 2a. Geschädigter hat keine gültige Police
 - 2a1. SicherMich informiert Geschädigten, weist Antrag zurück, protokolliert all dies und schließt den Vorgang
- 4a. Unfall verletzt wesentliche Versicherungsbedingungen
 - 4a1. SicherMich informiert Geschädigten, weist Antrag zurück, protokolliert all dies und schließt den Vorgang
- 4b. Unfall verletzt Details der Versicherungsbedingungen
 - 4b1. SicherMich tritt in Verhandlungen mit Geschädigtem über Verringerung der Zahlung ein

"Unfall" ist kein Akteur! Eigentlich ist gemeint:
"SicherMich stellt fest, dass ..." (Bedingung)
etc.



Zusätzliche Felder

Je nach Projektsituation ist es evtl. sinnvoll, der Use-Case-Schablone weitere Felder hinzuzufügen, z.B.

- Lieferant der Anforderungen
 - Name einer Person. Für die Rückverfolgung bei Rückfragen
- Trennung von Erweiterungen, Fehlerfällen, Alternativen
 - Werden ggf. an andere Use-Cases delegiert:
 - Erweiterungen: "extends"-Beziehung
 - Alternativen, Fehlerfälle: "generalizes"-Beziehung
- Besondere Anforderungen
 - z.B. betreffend Datenformate, GUI o.ä. Sparsam einsetzen!
- Offene Fragen
 - Streitpunkte, Zweifelspunkte, erwartete Ergänzungen etc.
- Annahmen
 - z.B. über Kenntnisse des Benutzers

Akteure und Rollen

- Die Akteure werden stets aus einer Perspektive bezeichnet, die zum Use Case gehört
 - z.B. Käufer, Geschädigter
 - jede Bezeichnung führt also eine Rolle ein
- Im Anforderungsdokument muss irgendwo zusätzlich festgelegt werden, welche Beteiligten welche dieser Rollen einnehmen können
 - und unter welchen Umständen
- Diese Festlegungen definieren Rechte, Benutzergruppen, Bündelung von Funktionen im GUI jeder Benutzergruppe etc.

Arten von Use Cases

Use Cases können für viele **Zwecke** eingesetzt werden:

- Beschreibung von **Geschäftsprozessen** einer Organisation
- Beschreibung der **funktionalen Anforderungen** einer Software oder Komponente
- Dokumentation des **Entwurfs** eines Systems
 - textuelle Alternative zu Sequenzdiagrammen
 - dann sind die Akteure Module und die Schritte sind z.B. Methodenaufrufe und deren Wirkungen/Resultate

Und in verschiedenen Projekt-**Kontexten**, insbesondere:

- Kleine, eng gekoppelte Gruppen von Beteiligten
- Große oder örtlich verteilte Gruppen von Beteiligten

Arten von Use Cases (2)

Deshalb sollte man Use Cases entlang 2 Dimensionen unterscheiden:

- Bezug (Anwendungsbereich, scope, "system under discussion", SuD)
 - eine Organisation Geschäftsprozess
 - ein Softwaresystem Interaktion, funktionale Anforderungen
 - eine SW-Komponente Verhalten einer Schnittstelle
- Abstraktionsgrad (Wie viele Details fehlen?)
 - Überblick hoher Abstraktionsgrad
 - Benutzerinteraktion mittlerer Abstraktionsgrad
 - Details geringer Abstraktionsgrad

Arten von Use Cases (3)

Anders ausgedrückt: Man muss beantworten

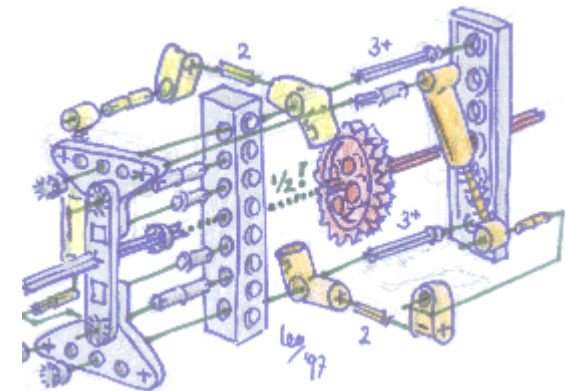
1. Von welchem System reden wir hier eigentlich?

- Der ganzen Firma, dem zu bauenden System, einem Teil des Systems oder noch was anderem
- Wenn das nicht klar ist, wird der Use Case unverständlich



2. Wie genau sollten wir die Abläufe beschreiben?

- Nur als Hintergrundinformation (zwecks Überblick)
- oder genau genug, um es konstruieren zu können
- Wenn die Genauigkeit *zu hoch* ist, liest niemand die Beschreibung mehr!



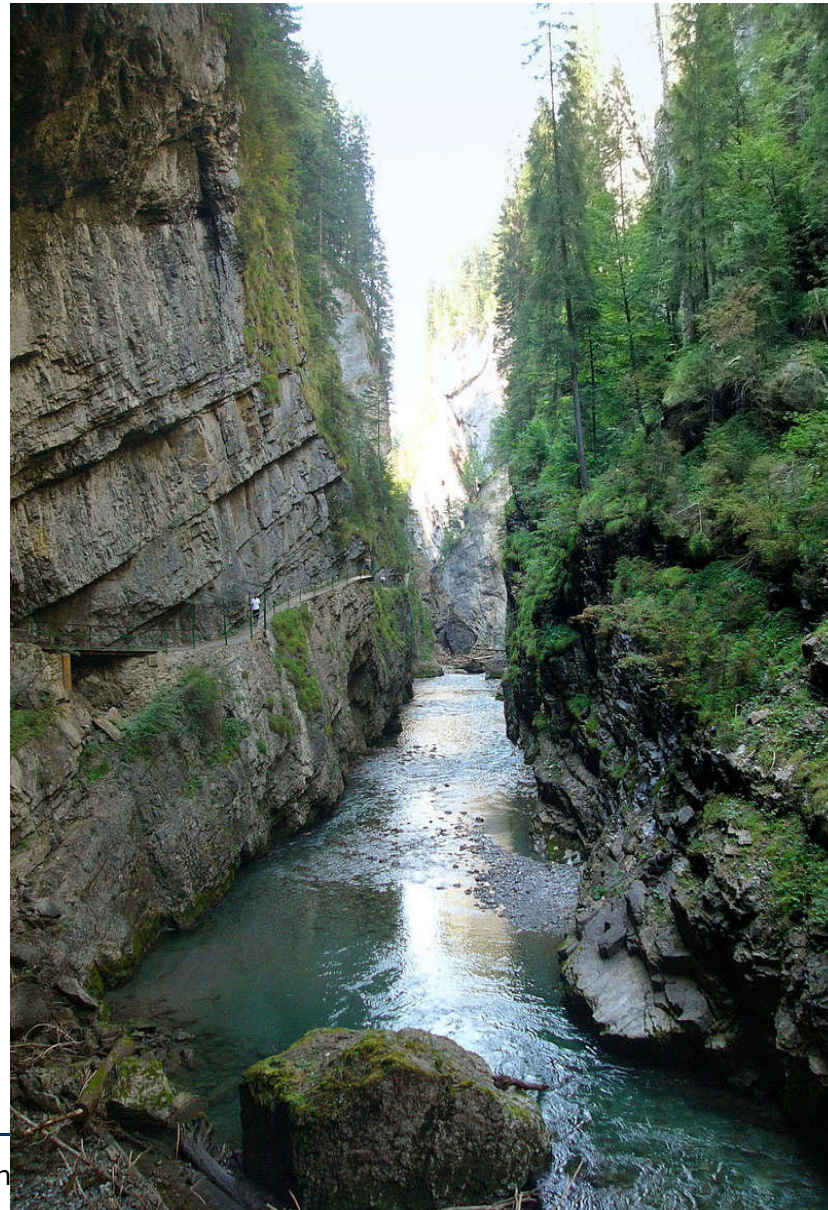
Arten von Use Cases (4)

- (1) Beispiel PAF (Aktien über WWW kaufen):
 - Bezug: Softwaresystem
 - Abstraktionsgrad: Benutzerinteraktion
- (2) Beispiel SicherMich (Auto-Unfallschaden ersetzt bekommen):
 - Bezug: Organisation
 - Abstraktionsgrad: Überblick
- Dies sind auch die beiden häufigsten Sorten:
 - (1) Interaktionsbeschreibungen für SW-Systeme liefern **funktionale Anforderungen**
 - (2) Überblicksbeschreibungen für Organisationen liefern den **Hintergrund für das Verständnis** von Anforderungen

Konventionelle vs. Agile Prozesse: Haltungen zu Use Cases

Use Cases?
Yeah!

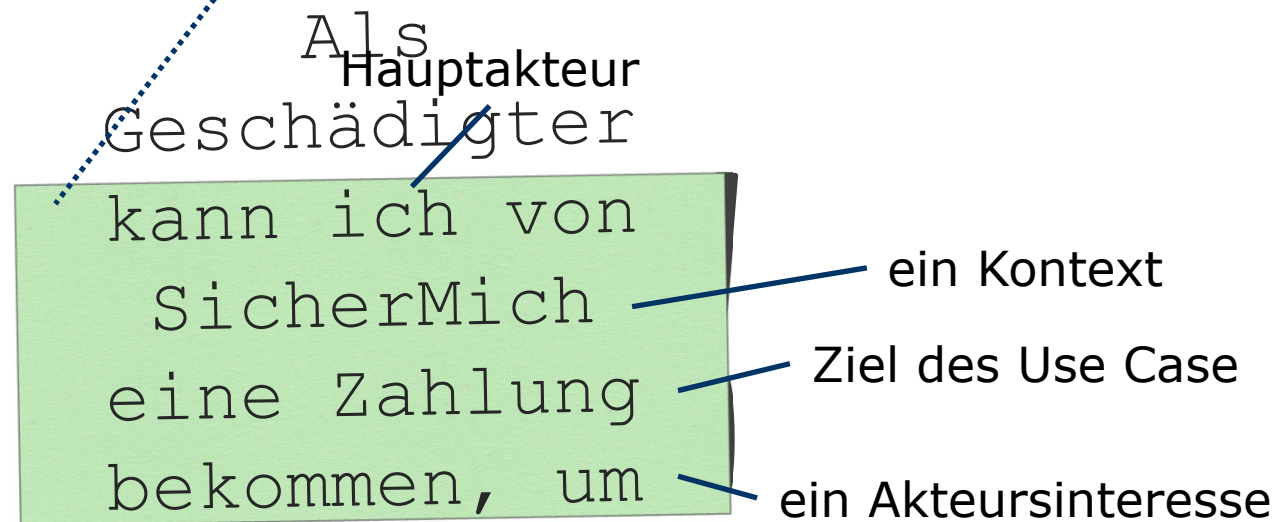
Use Cases?
Igitt!



Use Cases
sind ja sowas
von 1990er!

Konventionelle vs. Agile Prozesse: Typischer Umgang mit Use Cases

- Überblicks-Usecases??
- Normale Usecases: oft
- Whitebox: vielleicht mal zufällig
- Detail: selten
- Epics
- Story Cards
- "Guck in den Code!"
- "Notfalls bessern wir nach"



Vorsicht mit Präzision!

- Genauigkeit hat zwei Facetten:
Präzision (precision) und Richtigkeit (accuracy)
- $\pi \approx 4,141592654$
 - sehr präzise, aber wenig richtig
- $\pi \approx 3$
 - wenig präzise, aber völlig richtig
- $\pi \approx 3,14$
 - ausreichend präzise und völlig richtig
 - Der Fehler durch Präzisionsmangel ist hier schon kleiner als die Ungewissheit durch die Wärmeausdehnung!



Präzisions-Gefahren bei Use Cases

Wer zu früh zu genau spezifiziert, geht Gefahren ein:

- **Gefahr 1: Vieles falsch!**

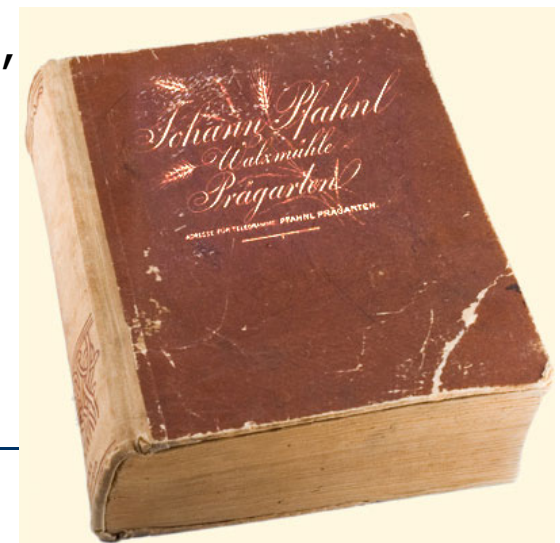
- Oft übersieht man anfangs etwas Wichtiges total oder sitzt einem wichtigen Missverständnis auf
 - und muss eigentlich alles ganz anders machen
- Wer dann schon in Details investiert hat, setzt viel Arbeit in den Sand

- **Gefahr 2: Niemand liest's!**

- Die Beteiligten an einem Projekt sind in der Regel sehr an Informationen, Überblicken etc. interessiert
- Aber wenn Dokumente zu umfangreich werden, liest niemand sie mehr

- Also: Weniger ist oft mehr

- Use Cases sollten immer nur gerade *genau genug* sein.



Präzision bei Use Cases: Empfehlung

Use Cases sollte man schrittweise präziser machen:

- Stufe 1: Hauptakteur und Ziel ("Story Card")
 - Schon das klärt wichtige Anforderungen und Beteiligte
 - Nun erst mal prüfen, ob alles wirklich stimmt
- Stufe 2: Haupt-Erfolgsszenario
 - Reicht in manchen Fällen schon ganz aus
 - Auch hier prüfen, ob es so akzeptabel ist (z.B. Benutzbarkeit)
- Stufe 3: Erweiterungsbedingungen (Varianten, Fehler)
 - Welche Probleme und Varianten müssen ggf. behandelt werden?
 - "Gefahr erkannt, Gefahr schon halb gebannt"
- Stufe 4: Erweiterungsschritte
 - Wie werden die Probleme/Varianten behandelt?
 - Kann man oft weglassen: Wird dann informell geklärt

Wichtigkeit von Zielen

- Ein gutes Anforderungsdokument benennt Anforderungen nicht einfach, sondern begründet sie
 - damit man Mehrdeutigkeiten leichter ausräumen kann
 - damit man Prioritäten vernünftig setzen kann
- In Use Cases werden die Begründungen durch die Ziele geliefert
 - Ein Ziel beantwortet zu den Schritten die Frage "Warum?"
 - Der Name eines Use Cases sollte ein Ziel bezeichnen
 - In einem umfassenderen Use Case tritt dieses Ziel als Schritt auf
- Ziele-als-Schritte beschreibt eine hierarchische Zerlegung
 - so dass einzelne Use Cases übersichtlich/beherrschbar bleiben
 - Ein guter Use Case hat meistens 3 bis 9 Schritte

Beispiel: Gesund werden (1)

- Use Case: Patient (Hauptakteur) will **gesund werden** (Ziel) und besucht dazu einen Arzt (SuD)
- Schritte:
 - 1. Arzt stellt Diagnose
 - 2. Arzt verordnet Medikament
 - 3. Patient beschafft Medikament (SuD: Apotheke)
 - 4. Patient nimmt Medikament ein
- Varianten:
 - 2a. Arzt überweist an Facharzt
 - 2b. Arzt überweist ins Krankenhaus

Beispiel: Gesund werden (2)

- Use Case:
Arzt (Hauptakteur) **stellt Diagnose** (Ziel) für Patient
 - Schritte:
 - 1. Arzt validiert Krankenversicherung via eGK
 - 2. Arzt befragt Patient nach Symptomen und Historie
 - 3. Arzt untersucht Patient
 - 4. Arzt entscheidet Diagnose(n)
 - 5. Arzt speichert Diagnose auf eGK
 - Varianten:
 - 1a. Arzt geht direkt zu Notfallversorgung über (Ende)
 - 1b. eGK oder eGK-Infrastruktur nicht verfügbar
 - 5a. Patient verweigert Speicherung
 - 5b. eGK oder eGK-Infrastruktur nicht verfügbar
- 

Beispiel: Gesund werden (3)

- Use Case: Arzthelfer (Hauptakteur)
validiert Krankenversicherung (Ziel) für Patient
über die eGK (SuD)
- Schritte:
 - 1. Arzthelfer legt HBA in eGK-Terminal ein und authentisiert sich
 - 2. Patient legt eGK in eGK-Terminal ein und authentisiert sich
 - 3. Arzthelfer fragt Krankenversicherungsstatus ab

Beispiel: Gesund werden – Beobachtungen

1. Es entsteht eine Hierarchie von Zielen/Zwecken:

- HBA in eGK-Terminal einlegen hat den Zweck
- Krankenversicherung validieren hat den Zweck
- Diagnose stellen hat den Zweck
- Gesund werden

2. "Arzt" ist gar keine Person, sondern eine Organisation

- Akteurswechsel von "Arzt prüft" zu "Arzthelfer legt ein"

3. Klare Benennung der Akteure macht auf Probleme aufmerksam:

- Wer wird am Ende die Diagnose speichern: Arzt oder Arzthelfer?
- Falls Arzt: Muss sich der Patient dann nochmals anmelden?
- Allgemein: Wie oft pro Tag melden sich Arzt/Arzthelfer an?
- etc.

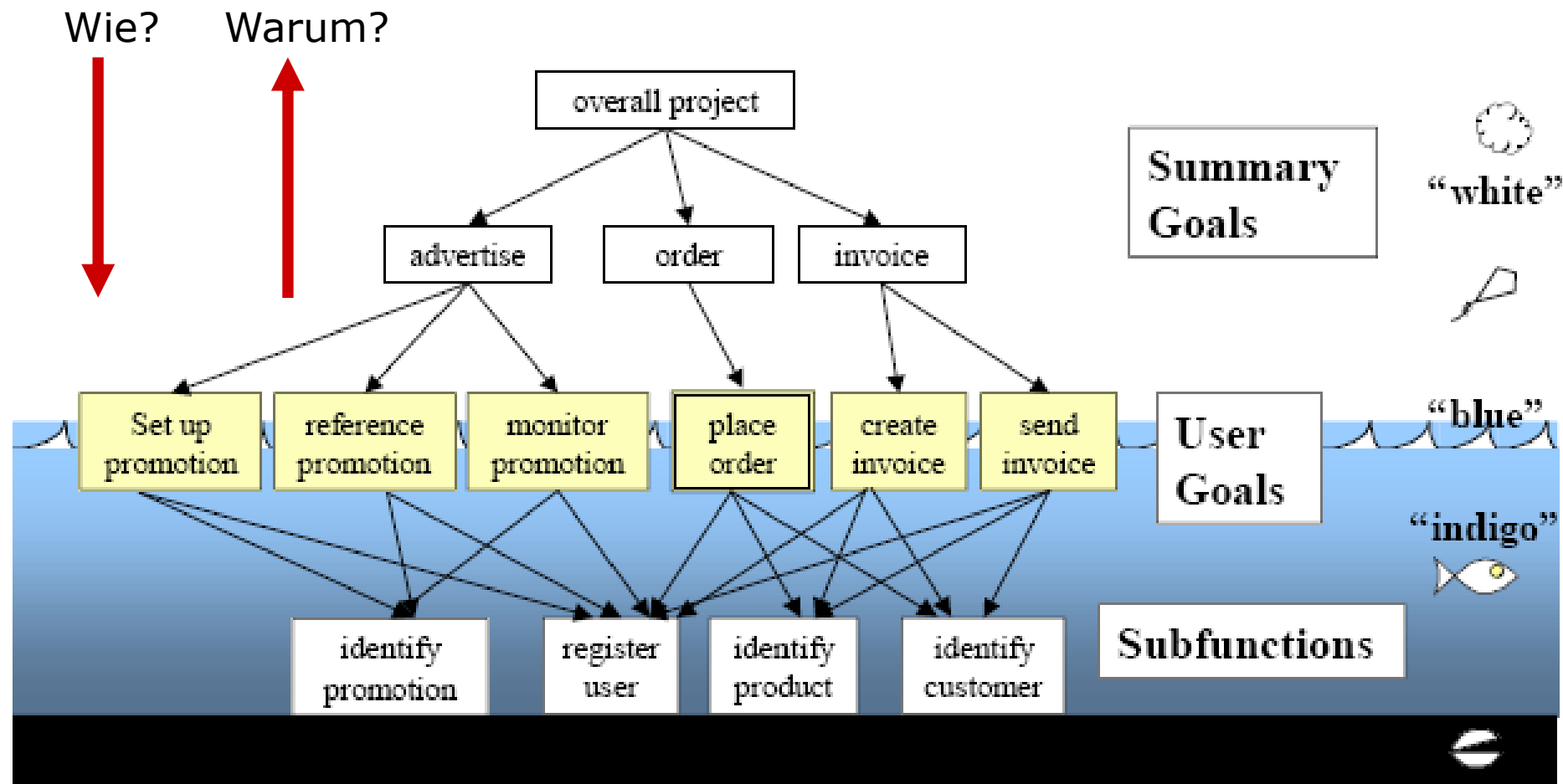
4. Geringe Zahl von Schritten erlaubt jederzeitige Orientierung

Nochmal: Das richtige Detailniveau

Ein gutes Projekt sollte haben:

- 1 Überblicks-Use-Case für das ganze Projekt
- Je 1 Überblicks-Use-Case für jeden Bereich von Zielen
 - ergibt meistens 1 bis 4; fast niemals mehr als ein Dutzend
- Je 1 **Überblicks-Use-Case** für Abläufe, die aus mehr als einer Sitzung am Rechner bestehen
 - selten mehr als ein Dutzend; oft Ersatz für obige Bereiche
- Je 1 **Benutzerziel-Use-Case** für jeden Ablauf, der "in einem Rutsch" (1 Sitzung) absolviert wird
 - bei einem großen System eventuell einige Dutzend
 - bei langen Abläufen zerlegt in Teile
- Je 1 **Detail-Use-Case** nur für besonders komplizierte Teilabläufe
 - Die dann oft mehrfach wiederverwendet werden
 - Je nach gewünschten Detailgrad viele oder wenige

Beispiel



Globale Schritte:

- Lege die Grenzen des Systems fest
 - Kontextbild, Liste von Eingaben und Ausgaben
- Finde alle Hauptakteure
 - Vollständigkeit ist essenziell
- Finde alle wichtigen Ziele jedes Hauptakteurs
 - *(Man kann die Leute dafür sogar fragen!)* 😊
- Schreibe wenige, grobe, zusammenfassende Überblicks-Use-Cases, die alle Ziele und Akteure erfassen
 - Das charakterisiert den Zweck und die Prioritäten des Systems
- Analysiere diese: Ziele zufügen, entfernen, vereinigen
 - Befrage dabei die Hauptakteure

Jetzt geht es an die einzelnen Benutzer-Interaktionen:

Schrittweises Modell für das Gesamtverfahren (2)

Für jeden Use Case auf Benutzerebene:

- Wähle nächsten Use Case zum Ausarbeiten aus
- Schreibe das Haupt-Erfolgsszenario
 - möglichst stets 3 bis 9 Schritte
- Finde und beschreibe alle Misslingensfälle und alternative Erfolgsfälle
- Breche Teil-Use-Cases heraus wo nötig

Use Cases und Szenarios, ATDD

- In Fällen, in denen das Schreiben eines Use Case schwer fällt
 - oder wenn man erwartet, dass er für manche Beteiligte zu schwierig zu verstehen ist (weil zu abstrakt)
- kann man zusätzlich Szenarios verwenden:
 - (meistens kommt man aber ohne Szenarios aus)
- Ein Szenario ist eine konkrete(re) Ausprägung eines Use Case
 - Es gibt also keine Varianten, sondern nur einen linearen Ablauf
 - Dieser kann ganz konkrete Datenwerte etc. angeben
 - Der Entwicklungsstil **ATDD** (Acceptance-test-driven development) würde statt 1 Use Case jeweils N konkrete Szenarios benutzen und diese als automatisierte Testfälle *vor* der SW implementieren
 - Diese Testfälle dienen als Spezifikation, wie sonst der Use Case.

Bei einem guten Use Case lautet jede Antwort "ja":

- Titel:
 - Aktive Verbalphrase, die das Ziel des Hauptakteurs nennt?
 - Ist das SuD für die Zielerreichung "zuständig"?
- Bereich (SuD):
 - Angegeben?
 - Wenn das SuD entworfen werden muss, müssen (a) alle Teile entworfen werden und (b) nichts außerhalb (Systemgrenze)?
- Detailgrad/Zielniveau:
 - Liegt das Ziel wirklich auf diesem Niveau?
 - Passt der Inhalt zum geplanten Detailgrad?
- Hauptakteur:
 - Hat er/sie/es Verhalten?
 - Hat er/sie/es ein Ziel, das zu einer Dienstzusicherung des SuD passt?

Checkliste für Use-Case-Bestandteile (2)

- Voraussetzungen:
 - Sind sie verbindlich und herstellbar?
 - Werden sie im Use Case nicht mehr überprüft?
- Beteiligte und ihre Interessen:
 - Muss das SuD in diesem Use Case diese Interessen bedienen?
- Mindestzusicherungen:
 - Sind die Interessen aller Beteiligten angemessen geschützt?
- Zusicherungen im Erfolgsfall:
 - Sind die Interessen aller Beteiligten befriedigt?
- Haupt-Erfolgsszenario:
 - Hat es 3 bis 9 Schritte?
 - Beschreibt es den Ablauf vom Auslöser bis zur Erfolgsgarantie?
 - Erlaubt es ggf. geeignete Abwandlungen in der Reihenfolge?

Checkliste für Use-Case-Bestandteile (3)

- Jeder Einzelschritt:
 - Ist er als erreichtes Ziel formuliert?
 - z.B. "validieren", nicht: "prüfen"
 - Treibt er den Prozess sichtbar voran?
 - Ist klar, wer der handelnde Akteur ist?
 - Ist die Absicht des Akteurs klar?
 - Abstrahiert der Schritt von der Bedienschnittstelle?
 - Ist erkennbar, welche Information verarbeitet wird?
- Erweiterungsbedingungen:
 - Kann das SuD diesen Fall entdecken (falls nötig)?
 - Muss das SuD diesen Fall behandeln?
- Inhalt des Use Cases insgesamt:
 - Gegenüber den Beteiligten: *"Ist dies, was Du möchtest?", "Kannst Du später entscheiden, ob es richtig gebaut wurde?"*
 - Gegenüber den Entwickler/inne/n: *"Kannst Du das bauen?"*

Und: Anforderungen müssen einleuchten

- Gute, sinnvolle Anforderungen kann man immer in einleuchtender Form beschreiben
 - Evtl. fehlt nur ein Überblick
- Hinterfragen Sie also uneinleuchtende!



Wie geht's dann weiter?

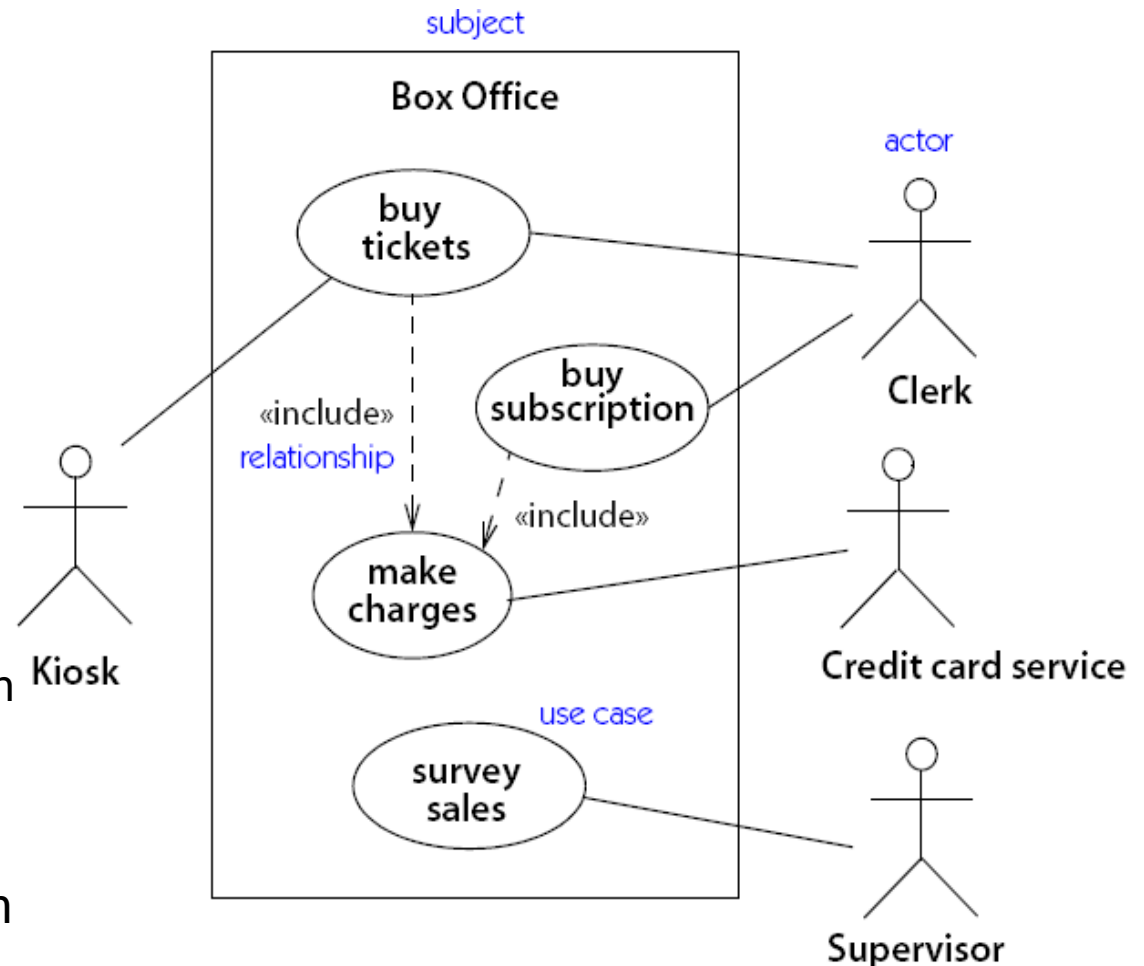
- Use Cases sind aus Benutzersicht wunderbar
 - klar, prüfbar, einleuchtend
- Auch als Werkzeug für die Anforderungserhebung sind sie günstig
 - einfach zu schreiben, einfach mit Beteiligten abzugleichen
- Aus Entwickler/innen/sicht lassen sie aber zu wünschen übrig
 - Sie beschreiben nicht, welche **Funktionen** die SW braucht
 - Und auch nicht, was für **Daten** im System vorhanden sind
 - Es bleibt unklar, wie man die Entwicklung auf Personen verteilen kann
- Deshalb müssen als nächstes Daten und Funktionen identifiziert werden
 - Anforderungs**analyse**
 - Siehe nächste Vorlesungen

Warnung: Use Cases sind nur ein Teil!

- Use Cases decken nur Verhaltensanforderungen ab!
- Sie beschreiben z.B. nicht
 - Domäneneigenschaften und Annahmen;
 - Geschäftsregeln (globale Festlegungen);
 - nichtfunktionale Anforderungen;
 - ggf. vorgegebenes Datenmodell, technische Schnittstellen u.ä.;
 - technische, organisatorische, rechtliche u.a. Randbedingungen
- Das Anforderungsdokument muss also mehr als nur Use Cases enthalten
 - aber Use Cases bilden einen guten Rahmen für den Rest

Warnung: Use-Case-Diagramme

- UML-Use-Case-Diagramme enthalten wenig Information
 - Geben Übersicht über Akteure und wenige Zusammenhänge zwischen Use Cases, sonst nichts
 - «include» war bei uns Unterstreichen
 - «extends» oder Generalisierung zeigen ausgelagerte Varianten und Fehlerfälle
 - Bitte nie das Diagramm mit dem Use Case verwechseln!

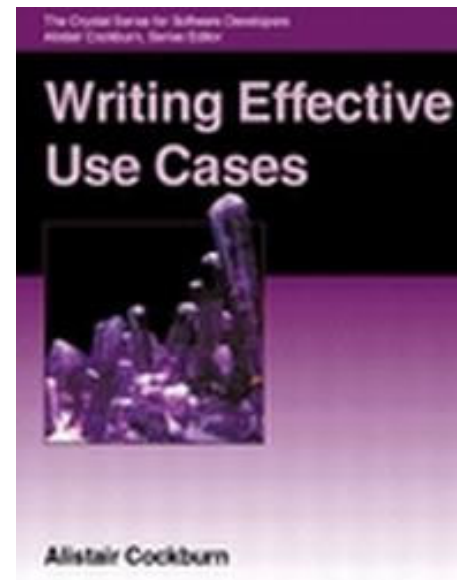


Zusammenfassung

- Was ist ein Use Case?
 - Beispiele
 - Felder
 - Arten von Use Cases
- Wichtige Parameter
 - Bereich ("system under discussion", SuD)
 - Detailgrad/Zielniveau
- Schrittweise Präzisierung
 - Gefahren von Präzision
- Gesamtvorgehens-Schritte
- Checkliste für Use Cases

Quelle:

- Alistair Cockburn: "Writing Effective Use Cases", Addison-Wesley 2001
 - Some related material:
<http://alistair.cockburn.us/Use+cases>



Danke!